Fox And Names (Codeforces 510C)

Ordenação topológica de letras com DFS

Alunos:
Samuel Ribeiro Benicio – 2310281
Adriel Medeiros Lins – 2013246
Manoel Sergio Costa Lima – 2422973
Orientador: Prof. Me. Ricardo Carubbi
Disciplina: Resolução prob Grafos
Centro de Ciências Tecnológicas – CCT
Universidade de Fortaleza – UNIFOR

Fox And Names

Roteiro da apresentação

Agenda

- Problema;
- Modelagem em grafo;
- Representação computacional;
- Solução com DFS;
- Validação e conclusão.

Problema

Fox And Names — Codeforces 510C

- **Lista de n** nomes (n ≤ 100, até 100 letras cada), na ordem dada
- **Achar uma ordem** das 26 letras que deixe a lista em ordem lexicográfica
- **Saída:** qualquer permutação válida, ou Impossible
- **Prefixo:** o nome menor deve vir primeiro

Entrada

```
4
baa
abcd
abca
cab
```

Uma saída válida

```
dbacefghijklmnopqrstuvwxyz
```

Modelagem em grafo

Uma condição de ordem vira um grafo de precedência

Vértices

Uma letra por vértice.

26 vértices, a até z.

Arestas

Nomes consecutivos: na 1ª posição com x ≠ y,

cria-se x → y.

Critério

Ciclo: contradição, Impossible.

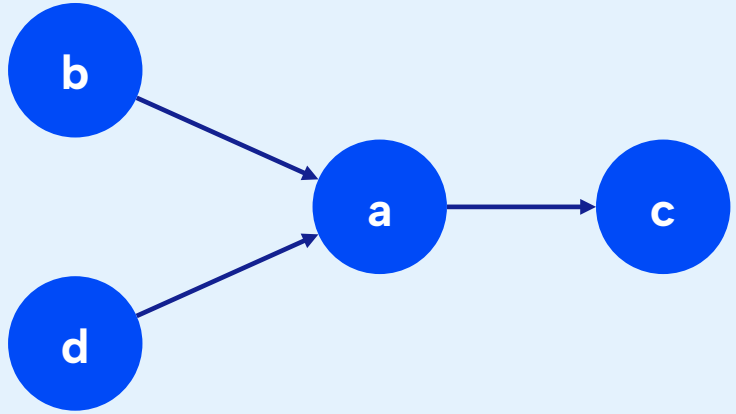
DAG: ordem topológica.

- **Prefixo:** sem aresta; o maior antes do seu prefixo é Impossible

Exemplo do grafo

Entrada: baa, abcd, abca, cab

- **baa × abcd:** b ≠ a, logo b → a
- **abcd × abca:** d ≠ a, logo d → a
- **abca × cab:** a ≠ c, logo a → c



As outras 23 letras ficam isoladas.

Representação computacional

Dígrafo de 26 vértices com listas de adjacência

- **Digraph:** 26 listas de adjacência
- **indice(c):** letra → índice, a=0 até z=25
- **existe:** matriz 26×26 evita aresta repetida
- **marked, on_stack:** vetores de tamanho 26 para a DFS

Listas do exemplo

```
adj(a) = [c]
adj(b) = [a]
adj(d) = [a]
adj(c) = []
adj(e..z) = []
```

Solução com DFS

Detectar ciclo e ordenar usam a mesma busca em profundidade

1. Arestas

Compara nomes consecutivos e cria x → y na 1ª diferença.

2. Prefixo

Nome maior antes do seu prefixo: Impossible.

3. Ciclo

DFS marca os vértices em processamento. Aresta para um deles é ciclo.

4. Ordem

Pós-ordem da DFS, invertida, é a ordem topológica.

- **Por que DFS:** Topological reúne as duas etapas na mesma busca
- **Tempo:** O(nL + V + E), com V = 26

DFS no exemplo

Entrada: baa, abcd, abca, cab

1. dfs(a)

visita c e termina

`post = [c, a]`

2. dfs(b)

a já concluído

`post = [c, a, b]`

3. dfs(d)

a já concluído

`post = [c, a, b, d]`

- **Saída:** dbacefghijklmnopqrstuvwxyz
- **Confere:** b → a, d → a e a → c são respeitadas
- **Ciclo:** com c → b, a DFS faria a → c → b → a; a ainda está em on_stack: Impossible

Validação

13 casos de teste e submissão aceita

- **Casos:** exemplos do enunciado, ciclos, prefixos, cadeia de 26 letras
- **Verificador:** confere permutação e ordem lexicográfica, pois a resposta não é única

13 / 13
casos corretos

Accepted
62 ms · 800 KB

Contest status

#	When	Who	Problem	Lang	Verdict	Time	Memory
393638964	Oct/07/2026 16:00 UTC-3	medeiros-cs	510C - Fox And Names	Python 3	Accepted	62 ms	800 KB

Conclusão

Uma condição de ordem vira um grafo de precedência

- **Modelagem:** 26 letras, aresta pela primeira diferença
- **Solução:** DFS para ciclo e ordem, O(nL + V + E)
- **Validação:** 13/13 testes e Accepted no Codeforces
- **Ideia central:** ciclo é contradição, DAG tem ordem

1. Modelar em grafo

2. Detectar ciclo

3. Ordem topológica